

Flutter CI CD Twelve Stage Study Guide

From Dart Defines and GetX to Secure Signed Release Artifacts

Project flutter_ci_cd

Professional study and revision document

8 October 2026

How to use this guide

This guide turns the project work into a practical learning path. Read Stages 1 through 12 in order the first time. Later, use the quick revision sections, command reference, and troubleshooting tables for fast review.

CI Continuous Integration: Automatically checks each code change through formatting, analysis, tests, coverage, and security gates.

CD Continuous Delivery: Produces a tested, versioned, releasable artifact. Deployment to Google Play is a later controlled step.

Main conclusion: A trustworthy release is not simply an APK that builds. It is source from the correct branch, validated by repeatable gates, configured for the intended environment, signed securely, versioned clearly, and retained as an auditable artifact.

Table of contents

- Stage 1 Analyze the current delivery process
- Stage 2 Build a professional CI foundation
- Stage 3 Design the testing strategy
- Stage 4 Enforce quality gates
- Stage 5 Measure test coverage
- Stage 6 Introduce DevSecOps
- Stage 7 Secure GitHub Actions and secrets
- Stage 8 Control dependency risk
- Stage 9 Maintain static quality and code health
- Stage 10 Manage environments with Dart Defines and flavors
- Stage 11 Separate validation testing and release builds
- Stage 12 Manage signed versioned artifacts
- Project architecture with GetX
- Release runbook and Git commands
- Future Google Play Internal Testing
- Troubleshooting quick reference
- Final revision checklist

The complete delivery map

```
feature branch -> pull request -> dev -> pull request -> main
                                   |
                                   +--> CI quality and security gates

manual request -> validate -> development or staging APK -> tester artifact

vX.Y.Z tag -> verify main and version -> quality gates -> production approval
            -> restore signing -> signed APK and AAB -> verify -> artifact
            -> stop before Google Play deployment
```

Workflow	Trigger	Purpose
flutter_ci.yml	PR or push to dev/main	Format, analyze, test, coverage, security, build verification
flutter_test_apk.yml	Manual workflow dispatch	Build a development or staging APK for testers
flutter_release.yml	v* tag or confirmed manual run	Create verified signed production APK and AAB

Stage 1 Analyze the current delivery process

Learning outcome: Understand the repository before changing automation.

What it is

Pipeline analysis maps the existing source branches, commands, tests, Android configuration, credentials, artifacts, and release decisions. This prevents automation from encoding incorrect assumptions.

Why it matters

A workflow can be syntactically valid and still release the wrong branch, wrong API URL, wrong flavor, or unsigned binary. Analysis defines the boundaries first.

Key concepts

- Repository truth: pubspec, Gradle files, tests, existing commands, and branch policy.
- Trust boundary: decide which data is public configuration and which data is secret.
- Release boundary: this project ends at signed downloadable artifacts, not Google Play deployment.
- Failure visibility: every failed step should point to a clear developer action.

Project baseline

Area	Current project decision
Branches	feature/* -> dev -> main
Flutter	Pinned to 3.47.5 in workflows
App version	1.0.1+2 in pubspec.yaml
Coverage	51.35 percent baseline with 50 percent gate
Release	Signed APK and AAB retained for manual QA

Common mistake: Starting with a large copied workflow before confirming the project commands and output paths.

Quick revision

- Inspect first, automate second.
- Define what the pipeline is allowed to release.
- A successful build alone is not a trustworthy release.

Stage 2 Build a professional CI foundation

Learning outcome: Understand workflow structure, triggers, jobs, steps, permissions, caching, and concurrency.

What it is

GitHub Actions reads YAML workflow files in .github/workflows. An event starts a workflow, a runner executes jobs, and each job executes ordered steps.

Term	Simple meaning
Workflow	The full automated process in one YAML file
Trigger	The event that starts it, such as pull_request or push
Runner	The temporary machine that executes a job
Job	A group of steps on one runner
Step	One action or shell command
Action	A reusable automation component
needs	A dependency that prevents a job from starting early
concurrency	Rules for cancelling or serializing related runs

Foundation pattern

```
name: Flutter CI
on:
  pull_request:
  push:
    branches: [dev, main]

permissions:
  contents: read

jobs:
  analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<immutable-commit-sha>
      - run: flutter pub get
      - run: flutter analyze
```

Tip: Pin Flutter and external actions. Reproducibility is more valuable than silently receiving a new toolchain during a release.

Quick revision

- Events start workflows; runners execute jobs; jobs contain steps.
- Use least privilege permissions.
- Use concurrency to avoid wasting runner time on obsolete commits.

Stage 3 Design the testing strategy

Learning outcome: Match each test type to its purpose and pipeline cost.

What it is

A testing strategy chooses the smallest reliable test for each risk. Fast deterministic tests run often; slower end-to-end tests run only when their environment is stable.

Test type	Purpose	Use in this project
Unit	A function, service, or controller in isolation	Configuration and HomeController behavior
Widget	A Flutter UI tree and user-visible state	Smoke rendering of the home screen
Integration	Multiple layers or a real app journey	Future controlled emulator suite

Test type	Purpose	Use in this project
Smoke	A small proof that critical startup works	Useful after a build or deploy
Regression	Prevents a known bug from returning	Added whenever a defect is fixed

Current tests

- Configuration and endpoint construction tests.
- GetX controller initial state, success, API failure, and timeout behavior.
- Widget smoke test for visible environment and API configuration.
- Nine tests passed during the latest project validation.

```
flutter test --reporter expanded
flutter test --coverage --reporter expanded
```

Warning: Do not add flaky hosted-emulator tests only to make the pipeline look advanced. First make data deterministic with mocks or a controlled staging backend.

Quick revision

- Test behavior, not implementation details.
- Use unit and widget tests on every pull request.
- Add integration tests when their environment is reliable.

Stage 4 Enforce quality gates

Learning outcome: Make downstream work depend on a single trustworthy pass or fail decision.

What it is

A quality gate combines required checks into an explicit decision. The build job declares needs: quality-gate, so it cannot run after a failed check.

Gate flow

```
format -----+
analyze -----+
test and coverage+--> quality and security gate --> build
secret scan ----+
dependency review
```

Fail fast order

1. Check formatting because it is fast and deterministic.
2. Analyze types, imports, and lints.
3. Run tests and enforce coverage.
4. Run security and dependency checks.
5. Build only after the gate succeeds.

Important: A gate should block only on meaningful failures. Noisy warnings teach developers to ignore automation.

Quick revision

- The gate is a decision, not another build.
- Use needs to enforce ordering.
- Run cheap checks before expensive builds.

Stage 5 Measure test coverage

Learning outcome: Use coverage as a trend and safety threshold, not as proof of correctness.

What it is

Line coverage: The percentage of executable source lines visited by tests.

Flutter writes LCOV data to `coverage/lcov.info`. The project tool reads this file and fails CI when coverage drops below 50 percent.

```
flutter test --coverage --reporter expanded
dart run tool/check_coverage.dart 50
```

Why 50 percent

The measured baseline was 51.35 percent, or 38 covered lines out of 74. A 50 percent threshold prevents immediate regression while leaving room to add meaningful tests gradually.

Good use	Bad use
Detect an unexpected coverage drop	Treat 100 percent as automatic quality
Find untested branches and error paths	Write weak assertions only to raise a number
Raise the threshold as tests improve	Include generated code without thought

Quick revision

- Coverage measures execution, not assertion quality.
- Protect the baseline, then improve it.
- Test important decisions and failure paths first.

Stage 6 Introduce DevSecOps

Learning outcome: Place security checks inside normal development instead of waiting for release day.

What it is

DevSecOps: The practice of making security a continuous responsibility across development, CI, release, and operations.

Security layers in this project

- Gitleaks scans repository history for credential patterns.
- A tracked-file policy rejects `.env`, keystore, `key.properties`, and service-account files.
- Dependency Review blocks newly introduced high or critical vulnerabilities in pull requests.
- Release signing material exists only on the temporary production runner.

If a secret leaks: Rotate or revoke the real credential first. Removing the text from the latest commit does not make the leaked credential safe.

Scope decision: CodeQL was not added because Dart is not a first-class CodeQL language for this project. Trivy was not added because there are no containers or infrastructure manifests to scan.

Quick revision

- Security belongs in every change, not only production.
- Scan history and protected files.
- Choose tools that match the repository technology.

Stage 7 Secure GitHub Actions and secrets

Learning outcome: Use least privilege, immutable actions, protected environments, and safe secret handling.

Configuration versus secret

Value	Where it belongs	Reason
API_BASE_URL	GitHub Environment variable	Public app configuration
WEB_BASE_URL	GitHub Environment variable	Public app configuration
Keystore base64	Production Environment secret	Private signing material
Key passwords	Production Environment secrets	Private credentials

Required production secrets

```
ANDROID_KEYSTORE_BASE64
ANDROID_KEYSTORE_PASSWORD
ANDROID_KEY_ALIAS
ANDROID_KEY_PASSWORD
```

Protected environment

The production build job declares environment: production. GitHub can pause that job for a required reviewer before it exposes production variables and signing secrets.

Warning: Dart Defines and mobile .env values are compiled into the app and can be inspected. Never place backend passwords, private tokens, or signing keys in them.

Action security: External actions are pinned to immutable commit SHAs. The version comment remains readable, while Dependabot proposes reviewed updates.

Quick revision

- Use variables for non-secret configuration and secrets for credentials.
- Protect production with required reviewers.
- Do not print or transform secrets unnecessarily.

Stage 8 Control dependency risk

Learning outcome: Keep packages and workflow actions current without merging updates blindly.

What it is

Dependency management combines automated update proposals with human review and CI evidence. Dependabot checks both pub packages and GitHub Actions every week.

Change	Meaning	Review approach
Patch 1.2.3 to 1.2.4	Usually fixes	Read notes and run all gates
Minor 1.2.3 to 1.3.0	Backward-compatible features by SemVer	Test affected behavior
Major 1.2.3 to 2.0.0	May contain breaking changes	Read migration guide and upgrade deliberately

Pull request dependency review

The dependency review job runs only for pull requests. The quality gate accepts skipped for non-PR events but rejects a failed review.

Important: Automation proposes dependency changes; it does not replace code review, release notes, or compatibility tests.

Quick revision

- Update regularly in small steps.
- Review the risk level of every update.
- Protect both app packages and CI actions.

Stage 9 Maintain static quality and code health

Learning outcome: Use formatting and analysis to keep feedback fast, consistent, and actionable.

What it is

Formatting removes style arguments. Static analysis catches type, import, API, and lint problems without running the app.

```
dart format --output=none --set-exit-if-changed lib test tool
flutter analyze
```

Local fix workflow

1. Run `dart format lib test tool` to apply formatting.
2. Run `flutter analyze` and fix the first clear cause.
3. Run `flutter test --coverage`.
4. Run the coverage checker before pushing.

Why separate jobs: Format, analyze, and tests can run in parallel and produce clearer failure messages. A final gate combines their results.

Quick revision

- Formatting is mechanical consistency.
- Analysis is compile-time feedback.
- Clear independent jobs reduce diagnosis time.

Stage 10 Manage environments with Dart Defines and flavors

Learning outcome: Understand defaults, .env files, CI variables, compile-time values, and native variants.

The three layers

Layer	Controls	Example
Dart Defines	Values compiled into Dart	APP_ENV and API_BASE_URL
Android flavor	Native application identity and build variant	dev, staging, production
GitHub Environment	CI variables, secrets, and approval policy	development, staging, production

Where the Gulf URLs are

```
// lib/config/app_config.dart
static const String apiBaseUrl = String.fromEnvironment(
  'API_BASE_URL',
  defaultValue: 'https://gulftest.traitsolutions.in/RestApi/app_api',
);

static const String webBaseUrl = String.fromEnvironment(
  'WEB_BASE_URL',
  defaultValue: 'https://gulftest.traitsolutions.in/RestApi/web_api/',
);
```

These are fallback values. They are used only when the build does not provide matching --dart-define values. The application does not call both a default API and an environment API. It resolves one final value during compilation.

Endpoint connection

```
AppConfig.apiBaseUrl
-> ApiEndPoint.baseUrl
-> '$baseUrl/attendance_list.php'
-> ApiService request
-> HomeController observable state
-> Obx user interface
```

Exercise 1 Dart Defines only

```
flutter run --dart-define=APP_ENV=staging --dart-define=API_BASE_URL=https://staging.example.com --dart-define=WEB_BASE_URL=https://staging.example.com/web_api/
```

Expected screen values: Environment: staging and API: https://staging.example.com.

Exercise 2 CI environment

```
env:
  APP_ENV: development
  API_BASE_URL: https://dev.example.com

- name: Build development APK
  run: >
    flutter build apk --release
    --dart-define=APP_ENV=${{ env.APP_ENV }}
    --dart-define=API_BASE_URL=${{ env.API_BASE_URL }}
```

Exercise 3 Native flavor plus Dart configuration

```
flutter run --flavor dev --dart-define=APP_ENV=development
flutter run --flavor staging --dart-define=APP_ENV=staging
flutter run --flavor production --dart-define=APP_ENV=production
```

.env and .env.example

.env.example is a committed list of safe sample names and values. .env is ignored and local. Flutter does not automatically read either file, and this exercise intentionally uses Dart Defines rather than a dotenv package.

Common mistake: Assuming a Windows environment variable or .env file automatically becomes available to `String.fromEnvironment`. It does not; pass `--dart-define` explicitly or through CI.

Quick revision

- Defaults are fallbacks, not a second API.
- Flavor controls native identity; Dart Defines control Dart values.
- GitHub Environment supplies CI policy and configuration.

Stage 11 Separate validation testing and release builds

Learning outcome: Use different workflows for code confidence, tester APKs, and production releases.

Why separate workflows

Need	Workflow	Retention and risk
Validate code changes	flutter_ci.yml	No distributable release; frequent and fast
Give testers an APK	flutter_test_apk.yml	Manual dev/staging artifact for 14 days
Prepare production	flutter_release.yml	Approved signed APK/AAB for 90 days

Testing APK flow

1. Open GitHub Actions and choose Build Testing APK.
2. Select development or staging.
3. Select debug for speed or release for production-like behavior.
4. The validate job runs formatting, analysis, tests, coverage, and secret scanning.
5. The build job reads environment variables and creates a versioned APK.
6. Download the artifact from the completed workflow run.

Production release flow

1. Verify the selected commit belongs to main history.
2. Derive the release version from a `v*` tag or confirmed manual input.
3. Compare the tag with `pubspec.yaml MAJOR.MINOR.PATCH+BUILD_NUMBER`.
4. Run analysis, tests, coverage, and secret scanning again.
5. Wait for production Environment approval.

6. Restore signing files on the temporary runner.
7. Build signed production APK and AAB.
8. Verify signatures and package versioned outputs.

Reason for retesting: A release is an important trust boundary. Running the gates in the release workflow proves the exact release source passed them, even if earlier CI results are old or missing.

Quick revision

- CI validates every change.
- Testing APKs are requested when humans need an installable build.
- Production release is versioned, approved, signed, and verified.

Stage 12 Manage signed versioned artifacts

Learning outcome: Produce traceable files that testers and release owners can verify.

What is an artifact

Artifact: A file produced by a workflow run and stored by GitHub Actions for download. It is not committed to Git.

Production package

```
app-production-v1.0.1+2.apk
app-production-v1.0.1+2.aab
SHA256SUMS.txt
release-info.txt
CHANGELOG.md
```

Signing lifecycle

1. Read encrypted values from the protected production Environment.
2. Decode the base64 keystore into android/app/release-keystore.jks.
3. Write android/key.properties with restricted permissions.
4. Build the production flavor.
5. Verify the APK with apksigner and the AAB with jarsigner.
6. Upload only the distributable package and metadata.
7. Remove restored signing files with an always-running cleanup step.

Version model

```
pubspec.yaml: 1.0.1+2
release tag: v1.0.1

1.0.1 = visible semantic version
2     = monotonically increasing Android build number
```

Checksum: SHA256SUMS.txt lets a consumer confirm that downloaded files are byte-for-byte identical to the files created by CI.

Current boundary: The workflow keeps the signed artifacts for 90 days but does not create a Play Store release.

Quick revision

- Version names explain the release; build numbers uniquely order it.
- Verify signatures after building.
- Retain checksums and release metadata with binaries.

Project architecture with GetX

The runtime path

```
GitHub variable or local --dart-define
-> AppConfig
-> ApiEndPoint
-> ApiService
-> HomeController
-> Obx widget
```

Configuration layer

AppConfig owns environment values. ApiEndPoint builds endpoint paths from the resolved base URL. Controllers and views do not decide which server to use.

Service and error layer

ApiService sends the request with GetConnect and converts unsuccessful HTTP responses into ApiException. Timeouts and unexpected errors are handled at the controller boundary so the UI receives a stable message.

Controller layer

```
final isLoading = false.obs;
final attendance = <dynamic>[].obs;
final errorMessage = ''.obs;

Future<void> loadAttendance() async {
  isLoading.value = true;
  errorMessage.value = '';
  try {
    attendance.assignAll(await apiService.getAttendanceList());
  } on ApiException catch (error) {
    errorMessage.value = error.message;
  } finally {
    isLoading.value = false;
  }
}
```

View layer

Obx listens to values created with .obs. When the controller changes isLoading, attendance, or errorMessage, only the reactive widget rebuilds.

Rule: Keep URL selection in AppConfig, HTTP details in ApiService, state transitions in the controller, and rendering in the view.

Release runbook and Git commands

Prepare a release

1. Update version in pubspec.yaml. Increase the build number for every store build.

2. Update CHANGELOG.md with user-visible changes.
3. Run all local validation commands.
4. Commit and push the release preparation to main through the protected branch flow.
5. Create an annotated tag that matches the pubspec visible version.
6. Push only the release tag and watch Flutter Release Preparation.
7. Approve the production Environment when the source and version are correct.
8. Download and manually test the signed artifact package.

```
dart format lib test tool
flutter analyze
flutter test --coverage --reporter expanded
dart run tool/check_coverage.dart 50

git status
git add pubspec.yaml CHANGELOG.md
git commit -m "chore(release): prepare version 1.0.1"
git push origin main

git tag -a v1.0.1 -m "Release version 1.0.1"
git tag
git push origin v1.0.1
```

Git commit style: Use short imperative messages such as test: add controller timeout coverage, ci: separate testing APK workflow, or chore(release): prepare version 1.0.1.

Do not: Do not tag a feature branch or change the tag after publishing. Correct the source, create a new version, and create a new tag.

Future Google Play Internal Testing

The next deployment stage should reuse the already tested signed AAB. It should not rebuild a different binary after approval.

Prerequisites

- Replace com.example.flutter_ci_cd with a permanent unique application ID.
- Create the application in Google Play Console.
- Configure Play App Signing.
- Create a least-privilege service account and enable the Android Publisher API.
- Store the service-account JSON as a protected GitHub secret.
- Start with the internal testing track and a manual approval.

```
signed release artifact
-> production approval
-> download the existing AAB
-> upload to Google Play Internal Testing
-> record Play release result
```

Security principle: The Play upload job should receive only the permissions and secret needed to upload the existing AAB.

Troubleshooting quick reference

Failure	Likely cause	First action
---------	--------------	--------------

Failure	Likely cause	First action
Format	Dart files changed format	Run dart format lib test tool
Analyze	Type, import, or lint issue	Run flutter analyze locally
Coverage	Meaningful paths are untested	Add behavior tests; rerun coverage
Gitleaks	Credential-like value exists in history	Rotate a real secret, then inspect finding
Dependency review	PR adds a known vulnerable package	Upgrade, remove, or review an exception
Environment	API variables are missing	Check selected GitHub Environment variables
Signing restore	Base64 or password is wrong	Recreate production Environment secrets
Tag validation	Tag differs from pubspec version	Match vX.Y.Z to X.Y.Z+N
Main verification	Tagged commit is outside main history	Merge to main before tagging
Gradle cache	Corruption or disk pressure	Inspect disk; clear only targeted recoverable caches

Final revision checklist

- I can explain CI, CD, workflow, trigger, runner, job, step, action, gate, secret, environment, flavor, Dart Define, and artifact.
- I know why the Gulf URLs live as safe defaults in AppConfig and how CI overrides them.
- I can trace configuration through ApiEndPoint, ApiService, GetX controller, and Obx.
- I can choose between CI validation, a manual testing APK, and a production release.
- I can explain why a release tag must match pubspec.yaml and belong to main.
- I know how signing files are restored, verified, cleaned, and never committed.
- I understand why coverage, secret scanning, dependency review, and action pinning are separate controls.
- I know the current workflow stops before Google Play and what is required to add Internal Testing safely.

One minute summary

The project uses Dart Defines for compile-time public configuration, Android flavors for native variants, and GitHub Environments for CI variables, secrets, and approval policy. GetX keeps configuration, HTTP work, observable state, and rendering in separate layers. Pull requests and protected-branch pushes run parallel quality and security checks. Developers request dev or staging APKs manually. A matching v* tag on main triggers fresh validation, production approval, temporary signing restoration, signed APK/AAB generation, signature verification, checksums, and a retained artifact. Google Play deployment remains a deliberate future stage.